A SYSTEMS-INSPIRED PORTFOLIO REDESIGN

A five-page design and implementation brief for adapting the visual language, interaction model and engineering storytelling of Karthick A's portfolio to Vikraman's existing website.

REFERENCE Karthick A · Embedded Systems Engineer	**EXISTING SITE** byvikraman.me · Next.js / TypeScript / Tailwind
DESIGN DIRECTION Technical instrument panel, component catalogue, engineering dossier	**PALETTE** Obsidian · Bone · Burnt Orange

THE CORE IDEA

Do not copy Karthick's identity or pretend Vikraman is already a professional embedded-systems product engineer. Adapt the **system-like presentation**: projects become engineered modules, technical details become visible evidence, and navigation feels like exploring a well-organised workbench.

Prepared as a practical redesign specification for use with Gemini / Google AI Studio. The brief assumes the existing website and its content system should be evolved, not discarded.

01 / WHAT MAKES KARTHICK'S SITE WORK

Reference: karthick-a.github.io/portfolio

The site presents itself as an engineering system rather than a conventional résumé. Its interface borrows from PCB layouts, component reference designators, test points, datasheets and instrument panels. That visual metaphor is consistent from the navigation to the project catalogue and contact section.

Visual and structural breakdown

- **System-console header:** compact identity, location, section links and a small power/status indicator. The 'pause motion' control acknowledges that animation is part of the interface and gives the visitor control.
- **Technical hero:** a clear role headline, a schematic-to-product lifecycle statement, a concise domain summary and direct actions for work and résumé.
- **Interactive project field:** projects are presented as labelled units (U1, U2, U3…) with a playful instruction to interact. This creates discovery without hiding the fact that the page is a portfolio.
- **Engineering evidence:** project entries pair concise descriptions with component/protocol tags and multiple proof images such as PCB renders, enclosures, bench tests, scope captures and field deployments.
- **Measured credibility:** a compact metrics strip gives scale and outcomes. These figures are specific to Karthick's work and must not be copied or imitated with invented numbers.
- **Layered engineer profile:** the About section is framed like a substrate / layer stack-up, turning skills into functional layers rather than a generic list of progress bars.
- **Test-point contact area:** email, phone, LinkedIn, GitHub and résumé are treated as labelled connection points. The metaphor carries through to the final section.

Why it feels distinctive

The strongest feature is not the dark colour scheme or the animations by themselves. It is the **consistency of the metaphor**. Typography, labels, project IDs, diagrams, tags and section names all reinforce one idea: this is a working engineer's system, opened up for inspection. The visual language gives the site a recognisable identity while the actual project evidence does the convincing.

What to borrow, and what not to borrow

Borrow: information hierarchy, technical labelling, project-as-module presentation, evidence-rich case studies, deliberate motion, status details and a strong visual concept. Do not borrow: Karthick's exact copy, professional metrics, client names, project claims, credentials, component lists, images, logo, or any personal information. Recreate the design logic around Vikraman's real work and current experience.

02 / TRANSLATING THE DESIGN TO VIKRAMAN

The goal is a systems-inspired portfolio that still feels like Vikraman. Keep the engineering interface, but allow room for software, electronics, photography, music and writing. Avoid turning the entire site into a fake industrial control dashboard.

Recommended identity and visual language

Use the chosen palette as the foundation: **Obsidian #191A18** for the main canvas, **Bone #E9E2D2** for primary text and light surfaces, and **Burnt Orange #C46B45** for active states, reference markers and carefully selected emphasis. Use muted warm greys for secondary labels and borders. Orange should not carry long paragraphs or tiny low-contrast text.

Suggested site architecture

- **Top navigation:** compact wordmark / name, Work, About, Notes and Contact. Keep the existing routes and URL structure where possible.
- **Hero:** name and ECE identity, one honest positioning line, a small technical motif (signal path, board trace, pin map or block diagram), and clear View Work / Résumé actions.
- **Selected Work:** a small number of featured project modules. Use stable project IDs and categories, but derive all titles, descriptions, links and media from existing MDX/frontmatter.
- **Work archive:** retain the existing unified /work page for projects and writing. Add filters or category navigation only if it improves discovery and works well on mobile.
- **Project detail:** an engineering dossier layout with overview, role/contribution, architecture, hardware/software, implementation, debugging, validation, results, learning and links. Sections must be optional, not forced.
- **About / capabilities:** a layered capability map grouped by programming, electronics, embedded experimentation, tools and creative practice. No arbitrary percentage skill bars.
- **Contact:** compact connection-point treatment for real contact and profile links.

Project modules for the content you actually have

Examples that can be presented honestly include the LPG Gas Leakage Detection System (Arduino Uno / MQ-6), 555 Timer Breadboard Piano, GARUDA disaster-response drone work (clearly describe team contribution), and ESP32-S3 / MPU6050 / OLED experiments. For each, distinguish what you personally designed, implemented, tested or learned from what the wider team completed.

Personal work belongs in the same system

Photography, guitar/music and film writing should not be buried or disguised as engineering projects. Give them a parallel ‘Beyond the Bench’ or ‘Field Notes’ treatment using the same typography and module language, but lighter visual density. The result should communicate a curious ECE student with multiple interests, not a one-dimensional embedded specialist.

03 / INTEGRATION WITH THE EXISTING WEBSITE

This is a redesign of the existing Next.js application, not a greenfield rebuild. Preserve the working content model, routes, metadata, deployment and existing project/blog material unless an actual technical reason requires a change.

Existing foundation to preserve

- **Stack:** Next.js App Router, TypeScript, Tailwind CSS, Framer Motion, MDX, shadcn/ui and Lucide, deployed on Vercel.
- **Routes:** keep /work, /projects/[slug] and /blog/[slug]. Do not break inbound links or rename slugs casually.
- **Content:** keep MDX as the source of truth. Do not duplicate project content in hardcoded React arrays or build a second CMS-like data layer.
- **Shared UI:** refactor existing navigation, buttons, typography, cards, tags, media and motion into reusable components where practical.
- **Quality:** preserve responsive behaviour, accessibility, reduced-motion support, SEO metadata, image optimisation, theme behaviour and current working interactions.

Suggested component translation

New design element	Implementation approach
System header	Adapt existing navbar; add subtle status / motion control only if useful and accessible.
Technical hero	Build from existing hero content; add a restrained SVG/CSS circuit or signal motif, not a heavy 3D dependency.
Project units	Create a reusable ProjectModule that receives existing MDX metadata and links to the current slug route.
Project dossier	Use existing MDX rendering pipeline; style headings, callouts, images, code and metadata as engineering documentation.
Layer stack-up	Render capability groups from a typed data source or current content model. Avoid unsupported proficiency scores.
Test-point footer	Restyle existing footer/contact links. Use actual destinations and provide accessible text labels.

Important implementation rule

Before changing code, inspect the repository structure, current components, MDX schema, route handlers, theme tokens, animation patterns and installed dependencies. Map the proposed design to what already exists. If a feature requires new data fields, make them optional and backwards-compatible, and document the exact frontmatter additions. Do not silently discard old content.

04 / MASTER IMPLEMENTATION PROMPT

Copy the prompt below into Gemini / Google AI Studio with the existing repository loaded. It instructs the model to study the reference and adapt its design system to the current application.

PROMPT

You are a senior product designer and frontend engineer. Redesign my existing personal portfolio using the design language and interaction ideas of Karthick A's embedded-systems portfolio: https://karthick-a.github.io/portfolio/. Study the reference carefully before coding. I love its technical, PCB-inspired identity, component-reference labels, project catalogue, evidence-rich engineering case studies, layered capability presentation and test-point contact section. I want an adaptation of its design logic, not a pixel-for-pixel clone.

```
MY SITE: https://byvikraman.me
REPOSITORY: https://github.com/V1kraman/portfolio-reimagined
STACK: Next.js App Router, TypeScript, Tailwind CSS, Framer Motion, MDX, shadcn/ui, Lucide, Vercel.
PALETTE: Obsidian #191A18, Bone #E9E2D2, Burnt Orange #C46B45.
```

PHASE 1: AUDIT BEFORE EDITING. Inspect the complete existing codebase. Identify routes, layout, shared components, MDX frontmatter/schema, content files, theme implementation, animation utilities, assets, metadata, responsive breakpoints and dependencies. Make a concise implementation map. Do not start by replacing the app or generating a new project.

PHASE 2: DESIGN TRANSLATION. Build a coherent systems-inspired design language: compact system header; editorial technical hero; restrained circuit / signal / PCB motifs; labelled project modules; evidence-led project pages; layered capability groups; and a test-point-inspired contact/footer. Keep the interface refined, readable and navigable. Use the palette consistently, with orange reserved for meaningful emphasis. Avoid generic cyberpunk styling, neon glows, excessive gradients, glassmorphism, random dashboard widgets and decorative clutter.

PHASE 3: CONTENT AND HONESTY. Preserve my actual identity as an Electronics and Communication Engineering student exploring software, electronics and embedded systems. Do not present me as an established professional embedded engineer. Do not invent metrics, clients, deployments, awards, skills, project outcomes or team ownership. For team projects, accurately separate my contribution from the team's work. Keep photography, music/guitar and film writing visible as part of my broader interests.

PHASE 4: ARCHITECTURE. Preserve /work, /projects/[slug] and /blog/[slug]. Keep MDX as the single source of truth. Build reusable components that consume the existing frontmatter/content. Do not hardcode featured project names or duplicate content in React arrays. Existing MDX files must continue to render. New metadata fields must be optional and backwards-compatible. Keep project and blog content in the existing unified /work experience.

PHASE 5: INTERACTION AND ACCESSIBILITY. Use motion purposefully and sparingly. Provide a pause/disable motion option only if it can be implemented robustly; always respect prefers-reduced-motion. Make interactive project modules keyboard accessible, with visible focus states and clear mobile fallbacks. Do not make essential content available only through hover, drag or 3D interaction. Ensure touch targets, contrast, semantic headings and screen-reader labels are appropriate.

PHASE 6: VALIDATION. Implement incrementally. Run the available lint, type-check and production build commands. Test the homepage, /work, at least one project detail route and one blog detail route at desktop and mobile widths. Check dark/light mode if currently supported, broken links, missing images, console errors, layout overflow and MDX compatibility. Fix regressions before finishing.

DELIVERABLE: A redesigned version of my existing site, not a separate demo. At the end, report the design decisions, files changed, any new optional MDX fields, commands run and validation results. List anything you could not verify. Do not claim tests passed unless they actually ran.

Acceptance checklist

- The result clearly evokes Karthick's technical systems aesthetic without copying his identity or content.
- Existing pages, slugs, MDX posts and project content still work.
- The homepage explains who Vikraman is within a few seconds and offers a clear path into work.
- Project pages show real process and evidence, not decorative technical jargon.
- Mobile users can access every essential feature without drag-only or hover-only interactions.
- The site remains fast, accessible, maintainable and deployable on the current Vercel setup.

Reference notes: Karthick A's public portfolio page was reviewed for the interface and content patterns described in this brief. Existing Vikraman stack, route and palette details are based on project context already established for this site. The live Vikraman site and repository could not be independently inspected in this document-generation session, so the prompt explicitly requires a fresh codebase audit before implementation.